Predicting Loan Default

Using only data known when the loan is approved

Phung Thi Thu Trang

DS66B, School of Technology, National Economics University

November 5, 2026

Outline

1. Problem and Data
2. Preprocessing
3. Modeling and Results
4. Experiments

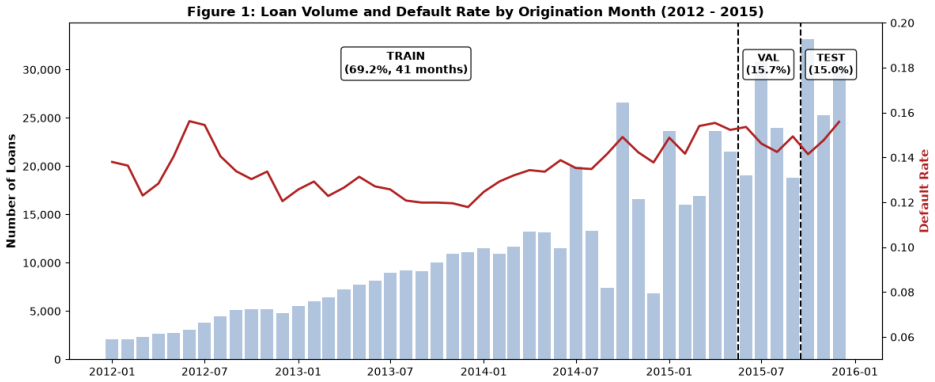


Figure: Loans by issue month and the train / validation / test split

Dataset and Goal

Dataset

- Lending Club accepted loans (US peer-to-peer lending)
- **2007 – 2018**: issue period
- **2,260,701** rows (loans)
- **151** columns

Goal: predict if a new loan will default

- Output: probability of default (0 – 1)
- Label: Charged Off = 1, Fully Paid = 0
- Input: only data known at approval

Steps: 1 Data Dictionary · 2 Target · 3 Split · 4 Cleaning · 5 Selection · 6 Engineering · 7 Modeling · 8 Test Results · + 3 Experiments

Problem Statement

Table: What the model does

Item	Description
Task	Binary classification (credit scoring)
Data	Accepted Lending Club loans, 2007 – 2018
Input	Only data known when the loan is approved: amount, grade, income, job, credit history...
Output	Probability of default, from 0 to 1
Label	1 = Charged Off, 0 = Fully Paid
Not used	Loans still running (Current, Late, In Grace Period): final result is not known yet

Most important rule: never use data created after approval (no data leakage)

Evaluation Metrics

- **ROC-AUC**: how well the model ranks bad loans above good loans
- **Gini** $= 2 \times \mathrm{AUC} - 1$
- **KS**: max gap between bad and good cumulative rates

No probability cut-off is needed: all three metrics measure ranking quality. The same function is used for every model and every data set.

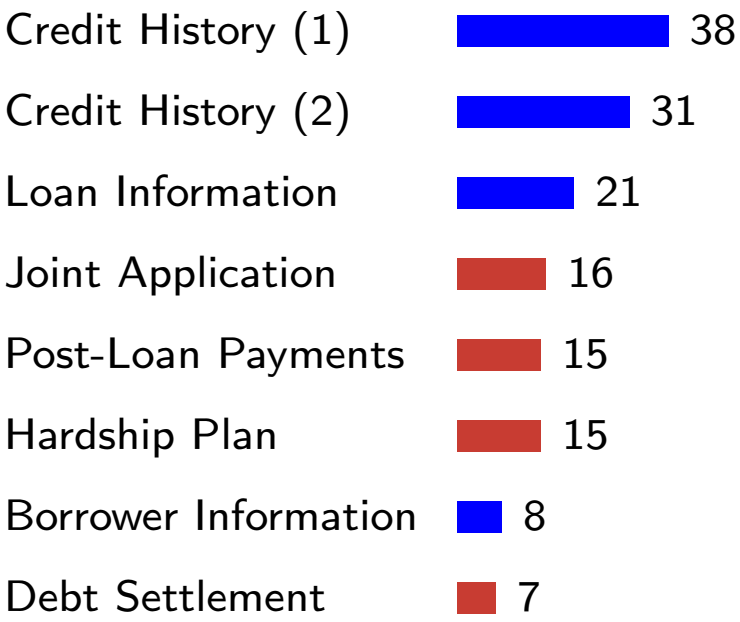
NB03 · cell 5

```
def calculate_metrics(y_true, proba):
    auc = roc_auc_score(y_true, proba)
    gini = 2 * auc - 1
    fpr, tpr, _ = roc_curve(y_true, proba)
    ks = np.max(tpr - fpr)
    return auc, gini, ks
```

Step 1: Data Dictionary

151 columns in 8 groups, each tagged: known at application?

Columns per group:



Red = groups only filled after the loan starts

Timing tag:

- **103** Yes: known at application
- **41** No: after loan or ID
- **7** Uncertain: timing not clear

151 / 151: dictionary matches the raw file header

Step 1 Details: 8 Column Groups

Table: Columns per group and their timing tag

Group	Columns	Yes	No	Uncertain
Loan Information	21	10	4	7
Borrower Information	8	8	0	0
Credit History (Part 1)	38	38	0	0
Credit History (Part 2)	31	31	0	0
Joint Application	16	16	0	0
Post-Loan Payments	15	0	15	0
Hardship Plan	15	0	15	0
Debt Settlement	7	0	7	0
Total	**151**	**103**	**41**	**7**

Each column gets a hypothesis: "is this known when the borrower applies?"
Notebook 02 checks this hypothesis again with code.

Step 1 Details: Yes / No / Uncertain

Yes (103): known at application. Borrower form or credit-bureau data; allowed as feature.

No (41): known only after the loan. Payments, hardship, settlement, IDs, label source; causes leakage.

id, member_id, url, loan_status, out_prncp, out_prncp_inv, total_pymnt, total_pymnt_inv, total_rec_prncp, total_rec_int, total_rec_late_fee, recoveries, collection_recovery_fee, last_pymnt_d, last_pymnt_amnt, next_pymnt_d, last_credit_pull_d, last_fico_range_high, last_fico_range_low, hardship_* (15 columns), debt_settlement_flag, debt_settlement_flag_date, settlement_status, settlement_date, settlement_amount, settlement_percentage, settlement_term

Uncertain (7): timing not clear. Not sure if known before approval; removed to be safe.

funded_amnt, funded_amnt_inv, issue_d, pymnt_plan, initial_list_status, policy_code, disbursement_method

Step 1 Details: Header Check

The dictionary must match the raw file header exactly: no missing, no extra, no duplicate columns.

NB01 · cell 30

```
missing_in_dictionary = [c for c in ALL_COLUMNS if c not in data_dictionary.index]
extra_in_dictionary = [c for c in data_dictionary.index if c not in ALL_COLUMNS]
assert not missing_in_dictionary and not extra_in_dictionary
```

Output

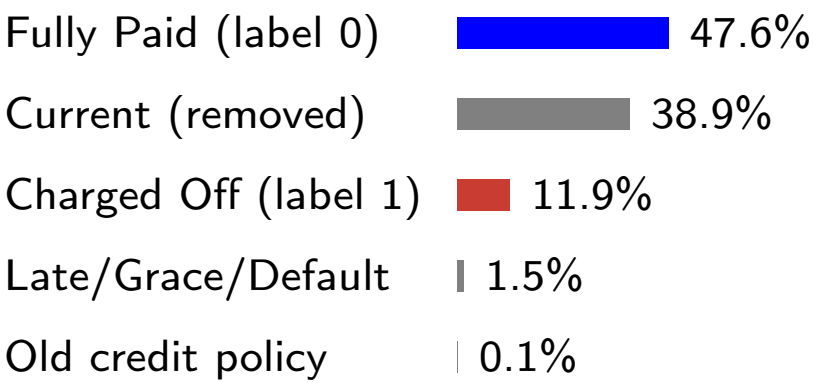
```
Raw columns missing from dictionary: []
Dictionary columns not in raw file: []
Duplicate columns in dictionary: 0
```

Step 2: Data Ingestion and Target Definition

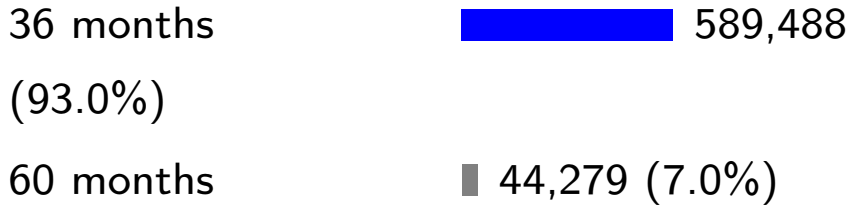
Only loans with a final result get a label

- **2,260,701** rows × 151 columns; **1,345,310** labelled rows
- Default rate among labelled loans: **20.0%**

Loan status (% of all rows):



Loan term (matured, from 2012):



Keep 36-month loans only, so all 3 sets share one term

Step 2 Details: Final Status Check

A finished loan has no principal left to pay. We compute the share of loans with `out_prncp` > 0 for each status.

Table: Share of loans with principal still owed

loan_status	Loans	Share with principal > 0
Fully Paid	1,076,751	0.000
Charged Off	268,559	0.000
Not meet credit policy: Fully Paid	1,988	0.000
Not meet credit policy: Charged Off	761	0.000
Current	878,317	0.995
Late (31-120 days)	21,467	0.999
In Grace Period	8,436	0.987
Late (16-30 days)	4,349	0.992
Default	40	1.000

4 statuses are final; 2 of them belong to an old credit policy (2,749 loans, 0.12%) and are removed.

Step 2 Details: Target Definition

NB01 · cell 53

```
LABEL_MAP = {"Fully Paid": 0, "Charged Off": 1}
df_raw["default"] = df_raw["loan_status"].map(LABEL_MAP)
# Output: Labelled row count: 1345310
```

Label balance (labelled rows):

0 = Fully Paid  1,076,751 · 80.0%

1 = Charged Off  268,559 · 20.0%

Step 2 Details: Data Types and Missing Values

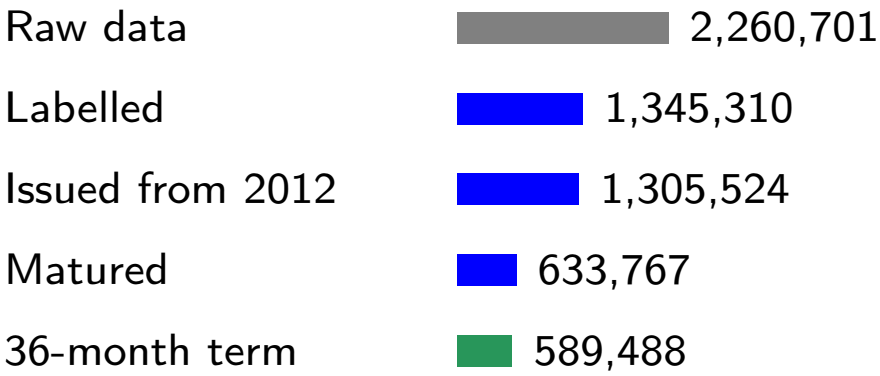
- **113** numeric columns (float64) · **38** text columns · **39** columns missing $> 80\%$
- 33 rows have no `loan_status` and no `issue_d`; they are removed

```
Rows: 2,260,701, Columns: 151
Memory usage: 3.29 GB
Loaded columns match dictionary columns exactly in the same order.
```

Step 3: Data Filtering and Out-of-Time Split

Train on old loans, test on newer loans: 70 / 15 / 15 by date

Filter funnel (rows):



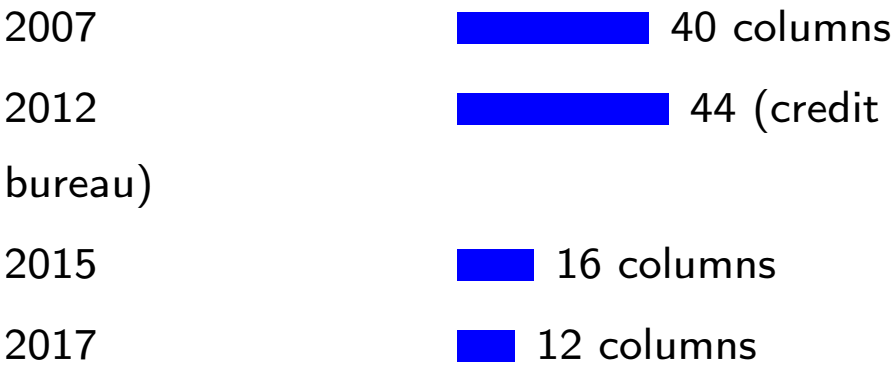
Three sets:

- **Train** 408,188 (69.2%): 2012-01 $\rightarrow$ 2015-05, default 13.7%
- **Val** 92,631: 2015-06 $\rightarrow$ 09, default 14.7%
- **Test** 88,669: 2015-10 $\rightarrow$ 12, default 14.8%

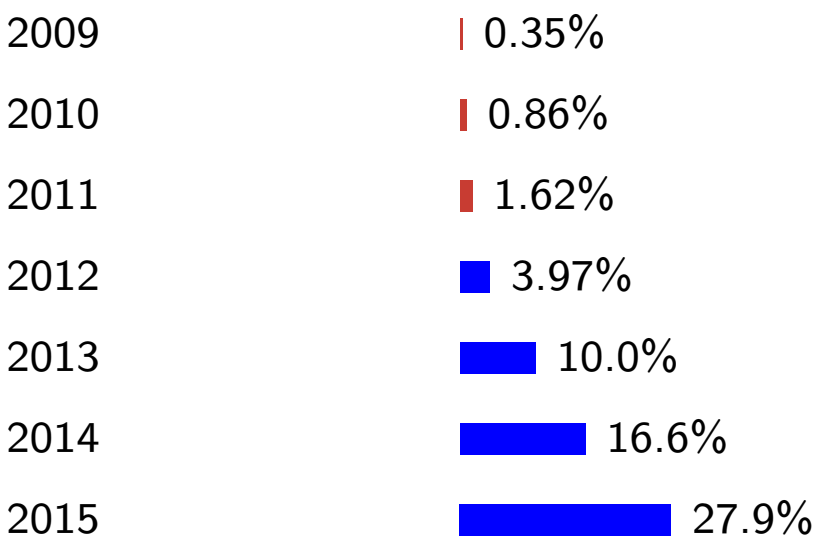
- **Matured**: issue month + term $\leq$ 2018-12 (99.6% of matured loans are labelled vs 42.4%)
- No month in two sets; default rate drifts up +1.1 points

Step 3 Details: Why Start in 2012?

Numeric columns by first year with data:



Share of labelled loans by year (%):



Before 2012: only 39,786 labelled loans (2.96%). From 2012: 44 credit bureau columns become available.

Step 3 Details: Matured Loan Rule

A loan is matured if issue month + term $\leq$ last month in data (2018-12). If the rule is right, almost all matured loans should have a final result.

NB01 · cell 77

```
def is_matured(issue_date, term_months, snapshot):
    end_month = month_number(issue_date) + term_months
    return end_month <= snapshot.year * 12 + snapshot.month
```

Table: Final result rate by group

Group	Loans	Resolved rate	Default rate
matured	676,459	0.996	0.148
unmatured	1,584,242	0.424	0.251

Step 3 Details: Filter Steps

NB01 · cell 87

```
steps = [
    ("Total raw data", pd.Series(True, index=df_raw.index)),
    ("Labelled (Fully Paid / Charged Off)", is_labelled),
    (f"... and issued from {START_YEAR}", is_labelled & from_start_year),
    ("... and matured", is_labelled & from_start_year & is_old_enough),
    ("... and 36-month term", is_labelled & from_start_year & is_old_enough & is_36_months),
]
```

Table: Rows after each filter

Step	Rows	% of raw	Step drop
Total raw data	2,260,701	100.0%	—
Labelled	1,345,310	59.5%	−40.5%
Issued from 2012	1,305,524	57.7%	−3.0%
Matured	633,767	28.0%	−51.5%
36-month term	589,488	26.1%	−7.0%

60-month loans from 2015 had no time to mature, so only 36-month loans are kept.

Step 3 Details: time_split Function

1. Sort by `issue_date`, then `id` · 2. Find cut dates at 70% and 85% of rows · 3. Split by date: a whole month always stays in one set

NB01 · cell 90

```
def time_split(df, train_share=0.70, val_share=0.15):
    df = df.sort_values(["issue_date", "id"]).reset_index(drop=True)
    cut_val = df["issue_date"].iloc[int(len(df) * train_share)]
    cut_test = df["issue_date"].iloc[int(len(df) * (train_share + val_share))]
    train = df[df["issue_date"] < cut_val]
    val = df[(df["issue_date"] >= cut_val) & (df["issue_date"] < cut_test)]
    test = df[df["issue_date"] >= cut_test]
    return train, val, test, cut_val, cut_test
# Output: Validation starts: 2015-06-01 / Test starts: 2015-10-01
```

Step 3 Details: Order Check and Split Result

```
assert train["issue_date"].max() < val["issue_date"].min()
assert val["issue_date"].max() < test["issue_date"].min()
assert len(train) + len(val) + len(test) == len(loans)
```

No val or test loan is older than the newest train loan: the model never sees the future during training.

Table: Three sets

Set	Rows	Share	From	To	Default rate
train	408,188	69.2%	2012-01	2015-05	0.137
val	92,631	15.7%	2015-06	2015-09	0.147
test	88,669	15.0%	2015-10	2015-12	0.148

All three sets are 100% 36-month loans (no term shift); default rate goes up over time (gap 0.011): temporal drift. Busiest month: 2015-10 with 33,186 loans. Known limit: 0.32% of 2015-12 loans have no final result yet.

Step 3 Details: Split Manifest

Saved to `artifacts/split_manifest.json` so other teams can check the same split.


```
{
  "start_year": 2012,
  "snapshot": "2018-12",
  "cut_val": "2015-06-01",
  "cut_test": "2015-10-01",
  "num_rows": {"train": 408188, "val": 92631, "test": 88669},
  "default_rate": {"train": 0.137, "val": 0.1474, "test": 0.1482},
  "md5_id_train": "8ce37fef31257778922ffe920c32511c"
}
```

Step 4: Data Cleaning, 6 Logic Rules

Each rule checks one row; the same rule runs on train, val and test

#	Logic rule (check)	Why	Handling
1	`annual_inc` ≤ 0	Income must be positive	Set to missing
2	`loan_amnt` ≤ 0	Amount must be positive	Set to missing
3	`dti` < 0 or > 100	DTI must be 0–100	Set to missing
4	`revol_util` > 100	Possible; extremes are noise	Cap at 102 (train p9
5	`fico_low` $>$ `fico_high`	Low must not pass high	Set both to missing
6	`earliest_cr_line` $>$ `issue_d`	Credit before the loan	Set to missing

Anti-leakage rules:

1. Split first, transform later
2. Median, mean, std, caps: **train only**
3. Pipeline `fit` on train; val and test only `transform`

Step 4 Details: Cleaning Rules (code)

Fixed rules applied row by row; the same function runs on train, val and test.

NB02 · fix_invalid_values (excerpt)

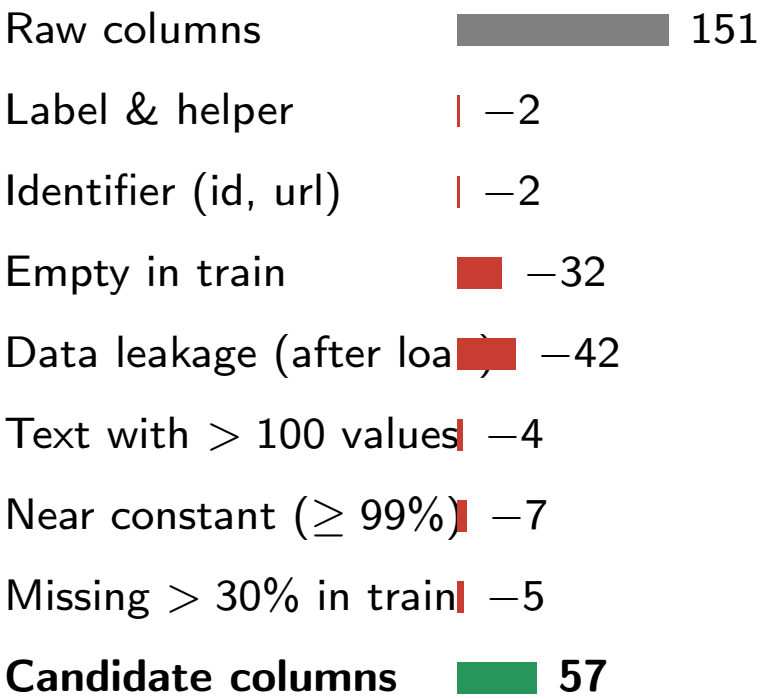
```
def fix_invalid_values(df):
    df = df.copy()
    df.loc[df["annual_inc"] <= 0, "annual_inc"] = np.nan
    df.loc[df["loan_amnt"] <= 0, "loan_amnt"] = np.nan
    df.loc[(df["dti"] < 0) | (df["dti"] > DTI_MAX), "dti"] = np.nan
    df["revol_util"] = df["revol_util"].clip(upper=REVOL_UTIL_CAP)
    bad_fico = df["fico_range_low"] > df["fico_range_high"]
    df.loc[bad_fico, ["fico_range_low", "fico_range_high"]] = np.nan
    first_credit = parse_month(df["earliest_cr_line"])
    df.loc[first_credit > df["issue_date"], "earliest_cr_line"] = np.nan
    return df
```

Step 4 Details: Leakage Rules

- Split first, transform later
- Anything learned from data (median, mean, std, one-hot categories, caps) is computed on train only
- Pipeline is fit on train; val and test only use transform
- EDA and feature choice use train only

Step 5: Feature Selection, 151 → 57

Each column goes through filters in a fixed order; first match removes it



Leakage check (single-column AUC, train):

- Best legitimate: `sub_grade` 0.6565
- `last_fico_range_low` 0.9074
- `recoveries` 0.9054
- `total_rec_prncp` 0.8280

Post-loan columns are far too strong

Step 5 Details: Filter Order (code)

Each column goes into exactly one group; the first "yes" answer decides the group.

NB02 · classify_column (labels translated)

```
def classify_column(column):
    p = profile.loc[column]
    if column in META_COLUMNS:            return "label / helper"
    if column in identifier_cols:         return "identifier"
    if column in empty_cols:              return "empty in train"
    if timing[column] in ("No", "Uncertain"): return "after loan / unclear timing"
    if column in joint_columns:           return "joint application"
    if column in many_values:             return "text with too many values"
    if p["top_value_share"] >= TOP_SHARE_CONSTANT: return "near constant"
    if p["missing_share"] > MAX_MISSING:  return "too many missing in train"
    return "keep as candidate"
```

Step 5 Details: Leakage Check (42 columns)

Single-column ROC-AUC on train shows how strong each column is alone. Post-loan columns are far too strong: they already "know" the result.

NB02 · single_column_auc (excerpt)

```
def single_column_auc(df, column):
    series = df[column]
    if pd.api.types.is_numeric_dtype(series):
        values = series.fillna(series.median())
    elif series.nunique() <= 100:
        values = df.groupby(series.fillna("(missing)"))["default"].transform("mean")
    else:
        values = series.notna().astype(int)
    auc = roc_auc_score(df["default"], values)
    return max(auc, 1 - auc)
```

Best legitimate column: `sub_grade` 0.6565; `last_fico_range_low` 0.9074, `recoveries` 0.9054, `collection_recovery_fee` 0.8677, `total_rec_prncp` 0.8280.

Step 5 Details: Removed Columns

After loan / unclear timing (42): funded_amnt, funded_amnt_inv, pymnt_plan, initial_list_status, out_prncp, out_prncp_inv, total_pymnt, total_pymnt_inv, total_rec_prncp, total_rec_int, total_rec_late_fee, recoveries, collection_recovery_fee, last_pymnt_d, last_pymnt_amnt, last_credit_pull_d, last_fico_range_high, last_fico_range_low, policy_code, hardship_* (15 columns), disbursement_method, debt_settlement_flag, debt_settlement_flag_date, settlement_status, settlement_date, settlement_amount, settlement_percentage, settlement_term

Label / helper (2): issue_d, loan_status · **Identifier (2):** id, url

Empty in train (32): member_id, next_pymnt_d, annual_inc_joint, dti_joint, verification_status_joint, open_acc_6m, open_act_il, open_il_12m, open_il_24m, mths_since_rcnt_il, total_bal_il, il_util, open_rv_12m, open_rv_24m, max_bal_bc, all_util, inq_fi, total_cu_tl, inq_last_12m, revol_bal_joint, sec_app_* (12 columns)

Text with $>$ 100 values (4): emp_title, desc, title, zip_code

Near constant $\geq$ 99% (7): collections_12_mths_ex_med, application_type, acc_now_delinq, chargeoff_within_12_mths, delinq_amnt, num_tl_120dpd_2m, num_tl_30dpd

Missing $>$ 30% in train (5): mths_since_last_delinq, mths_since_last_record, mths_since_last_major_derog, mths_since_recent_bc_dlq, mths_since_recent_revol_delinq

Step 5 Details: 57 Candidate Columns

loan_amnt, term, int_rate, installment, grade, sub_grade, purpose, emp_length, home_ownership, annual_inc, verification_status, addr_state, dti, delinq_2yrs, earliest_cr_line, fico_range_low, fico_range_high, inq_last_6mths, open_acc, pub_rec, revol_bal, revol_util, total_acc, tot_coll_amt, tot_cur_bal, total_rev_hi_lim, acc_open_past_24mths, avg_cur_bal, bc_open_to_buy, bc_util, mo_sin_old_il_acct, mo_sin_old_rev_tl_op, mo_sin_rcnt_rev_tl_op, mo_sin_rcnt_tl, mort_acc, mths_since_recent_bc, mths_since_recent_inq, num_accts_ever_120_pd, num_actv_bc_tl, num_actv_rev_tl, num_bc_sats, num_bc_tl, num_il_tl, num_op_rev_tl, num_rev_accts, num_rev_tl_bal_gt_0, num_sats, num_tl_90g_dpd_24m, num_tl_op_past_12m, pct_tl_nvr_dlq, percent_bc_gt_75, pub_rec_bankruptcies, tax_liens, tot_hi_cred_lim, total_bal_ex_mort, total_bc_limit, total_il_high_credit_limit

Step 6: Feature Engineering and Preprocessing, 57 → 153

New features, remove duplicates, then impute, scale and one-hot in one pipeline

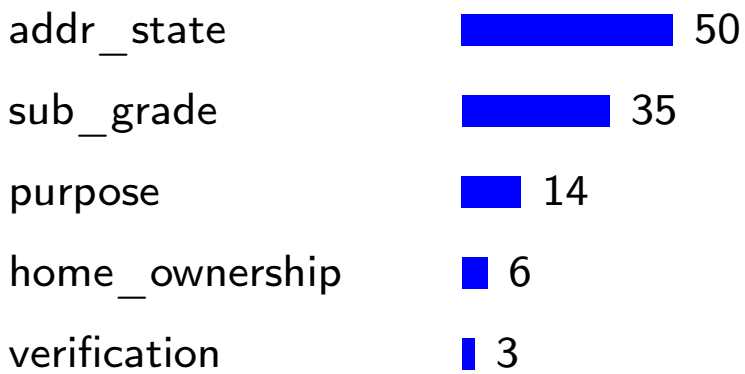
57	→	+1	→	+6	→	−14	→	50
candidates		issue_date (anchor)		new features		duplicates & high corr.		45 num + 5 cat

→ **153** final matrix

6 new features:

- `fico_avg`: mean of FICO range
- `credit_history_years`: years of credit
- `loan_to_income`: loan / income
- `log_annual_inc`, `log_revol_bal`: fix skew
- `emp_length_years`: text → number

One-hot columns (108 total):



45 numeric: median impute + StandardScaler

Step 6 Details: 6 New Features

Feature	Formula	Note
fico_avg	(fico_range_low + fico_range_high) / 2	FICO is a 4-point range; the tw
credit_history_years	(issue_date − earliest_cr_line) / 365.25	Years of credit history at applica
loan_to_income	loan_amnt / annual_inc, cap 0.5	Loan size vs yearly income (cap
log_annual_inc	log(1 + annual_inc)	Income is very right-skewed
log_revol_bal	log(1 + revol_bal)	Card balance is right-skewed; lo
emp_length_years	"< 1 year" → 0, "10+ years" → 10	Text to number

NB02 · feature functions (excerpt)

```
def make_fico_avg(df):
    return (df["fico_range_low"] + df["fico_range_high"]) / 2

def make_credit_history_years(df):
    first_credit = parse_month(df["earliest_cr_line"])
    return (df["issue_date"] - first_credit).dt.days / 365.25

def make_emp_length_years(df):
    text = df["emp_length"].replace({"< 1 year": "0 years"})
    return text.str.extract(r"(\d+)")[0].astype(float)
```

Step 6 Details: 14 Removed Columns

- −4 Replaced by a new feature: fico_range_low, fico_range_high, annual_inc, revol_bal
- −2 Text changed to number: emp_length, earliest_cr_line
- −2 Same info as sub_grade: grade, int_rate
- −6 $|r| \geq 0.90$ pairs + date anchor: issue_date, and 5 high-correlation columns

For each pair with $|r| \geq 0.90$, keep the column with higher single-column AUC. `term` is constant (100% 36 months), so `term_months` is not created.

NB02 · high_corr_pairs

```
def high_corr_pairs(corr_matrix, threshold):
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape, dtype=bool), k=1)).stack().dropna()
    return upper[upper.abs() >= threshold].sort_values(key=abs, ascending=False)
```

Step 6 Details: Why Median and Scaling?

- **31** numeric columns have missing values in train
- Most are right-skewed, so the mean is pulled up by outliers (mean is up to **+138%** above the median)
- Example: `bc_open_to_buy` mean is 138.2% above median; `avg_cur_bal` 100.7%
- Median keeps the center of typical borrowers
- Scales differ by $>$ 100,000$\times$: from 0.3 (`delinq_2yrs`) to 127,000 USD (`tot_cur_bal`)
- StandardScaler ($\mu = 0$, $\sigma = 1$) helps the L2 penalty treat all features fairly

Step 6 Details: Pipeline (code)

NB02 · pipeline (excerpt)

```
def make_preprocessor(numeric_cols, categorical_cols):
    numeric_part = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ])
    categorical_part = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
    return ColumnTransformer([
        ("num", numeric_part, numeric_cols),
        ("cat", categorical_part, categorical_cols),
    ])

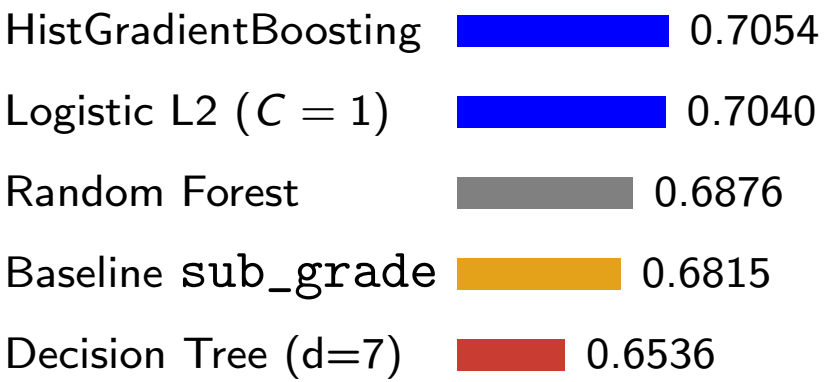
full_pipeline = Pipeline([
    ("clean", FunctionTransformer(fix_invalid_values)),
    ("features", FunctionTransformer(add_features)),
    ("prep", make_preprocessor(NUMERIC_FINAL, CATEGORICAL_FINAL)),
])
full_pipeline.fit(train)          # fit on train only
X_val = full_pipeline.transform(val)
```

Output: X_train = 408,188 × 153.

Step 7: Model Training and Validation

5 models tuned with TimeSeriesSplit; compared on the validation set

Validation ROC-AUC (axis starts at 0.60)



Overfitting (train − val AUC):

Random Forest +0.018; HistGB +0.004; Logistic −0.017

Bootstrap 95% CI of ΔAUC vs baseline:

Model	95% CI
HistGB	[+0.021, +0.027]
Logistic L2	[+0.020, +0.025]
Random Forest	[+0.002, +0.010]
Decision Tree	[−0.033, −0.023]

Step 7 Details: Hyperparameter Tuning

GridSearchCV with `TimeSeriesSplit(n_splits=3)` inside train; score: ROC-AUC; no shuffled K-fold.

Table: Best setting per model (CV ROC-AUC on train)

Model	Grid	Best	CV AUC
Logistic L2	$C \in \{0.001 \ldots 10\}$	$C = 1.0$	0.6855
Decision Tree	max_depth 3–10	depth = 7	0.6355
Random Forest	trees 50/100/150, depth 6/8/10, leaf 30/50/60	150, 10, 50	0.6741
HistGradientBoosting	lr 0.05–0.20, depth 4–10, iter 50–150	0.05, 10, 150	0.6852

NB03 · cell 8

```
tscv = TimeSeriesSplit(n_splits=3)
grid_lr = GridSearchCV(
    LogisticRegression(max_iter=1000, random_state=SEED, solver="lbfgs"),
    {"C": [0.001, 0.01, 0.1, 1.0, 10.0]},
    cv=tscv, scoring="roc_auc", refit=False, n_jobs=4,
)
grid_lr.fit(X_train, y_train)
```

Limit: RF and HistGB best values sit on the grid edge; gains between nearby settings are $\sim$0.001.

Step 7 Details: Baseline sub_grade

Lending Club already has its own grade. Baseline score = default rate of each sub_grade in train. A model must beat this to be useful.

NB03 · cell 7

```
subgrade_risk = (train_meta.assign(target=y_train)
                 .groupby("sub_grade")["target"].mean().to_dict())
p_val_base = val_meta["sub_grade"].map(subgrade_risk).values
```

Step 7 Details: Overfitting Check

Table: Train vs validation (val = 92,631 loans)

Model	AUC train	AUC val	Gap	Gini val	KS val
Baseline sub_grade	0.6565	0.6815	−0.0250	36.31%	27.02%
Logistic L2 ($C = 1$)	0.6873	0.7040	−0.0167	40.79%	29.94%
Decision Tree (depth 7)	0.6541	0.6536	+0.0006	30.71%	22.25%
Random Forest (150, depth 10)	0.7060	0.6876	+0.0184	37.52%	27.21%
HistGradientBoosting	0.7096	0.7054	+0.0042	41.07%	30.02%

Random Forest has the largest gap: light overfitting. Val higher than train (baseline, logistic) is observed; the cause was not tested.

Step 7 Details: Validation Curves

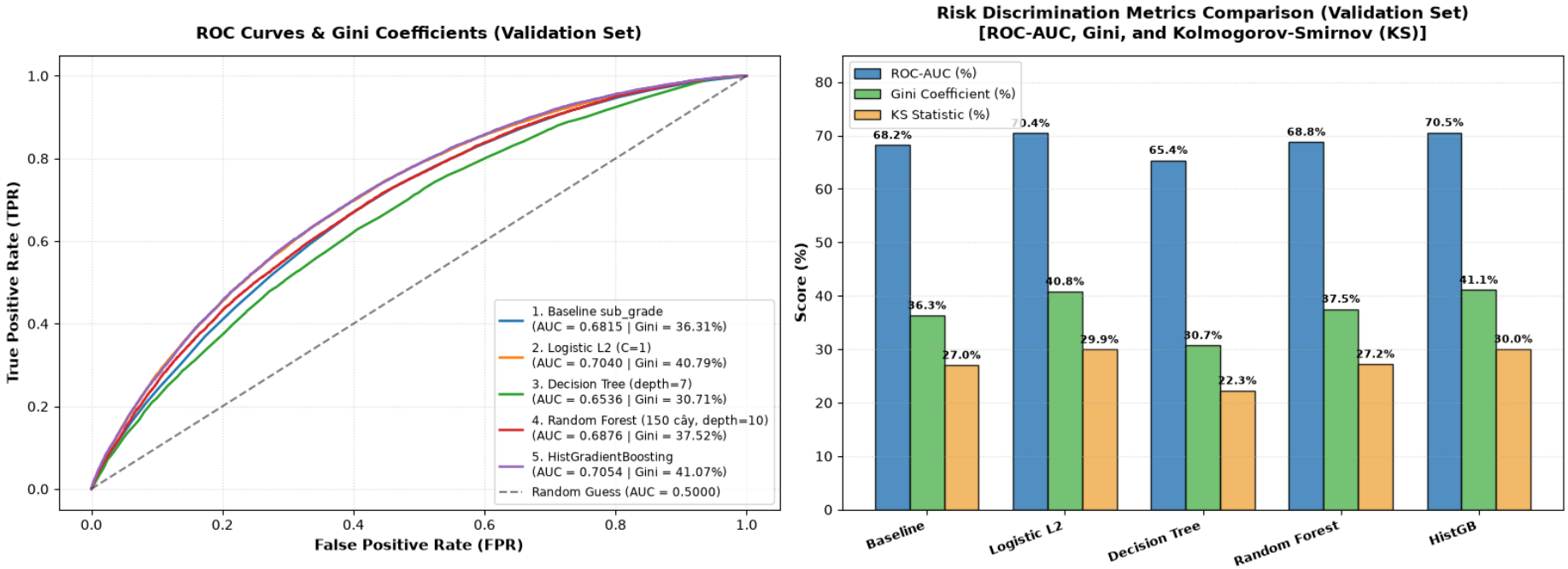


Figure: ROC curves and metric comparison on validation (notebook 03 output)

Step 7 Details: Bootstrap 95% CI

$B = 1{,}000$ resamples of the validation set. If the 95% CI does not contain 0, the gain is real.

NB03 · cell 21 (excerpt)

```
for _ in range(B):
    idx = np.random.randint(0, n_val, size=n_val)
    auc_m = roc_auc_score(y_val[idx], m["p_va"][idx])
    auc_b = roc_auc_score(y_val[idx], p_val_base[idx])
    diffs.append(auc_m - auc_b)
ci_lower, ci_upper = np.percentile(diffs, [2.5, 97.5])
```

Table: ΔAUC vs baseline (validation)

Model	ΔAUC	95% CI	Better than baseline?
Logistic L2	+0.0225	[+0.0196, +0.0253]	Yes
Decision Tree	−0.0280	[−0.0329, −0.0233]	No
Random Forest	+0.0061	[+0.0024, +0.0102]	Yes
HistGradientBoosting	+0.0238	[+0.0206, +0.0270]	Yes

Step 8: Out-of-Time Test Results and Conclusion

Test set used once: 88,669 loans, 2015-10 to 2015-12

Table: Out-of-time test performance

Model	ROC-AUC	Gini	KS
Baseline `sub_grade`	0.6818	36.4%	26.5%
Logistic L2 (chosen)	**0.7039**	40.8%	29.9%
HistGradientBoosting	0.7055	41.1%	30.0%

HistGB − Logistic (95% CI): $[-0.0002, +0.0031]$ contains 0, no real difference

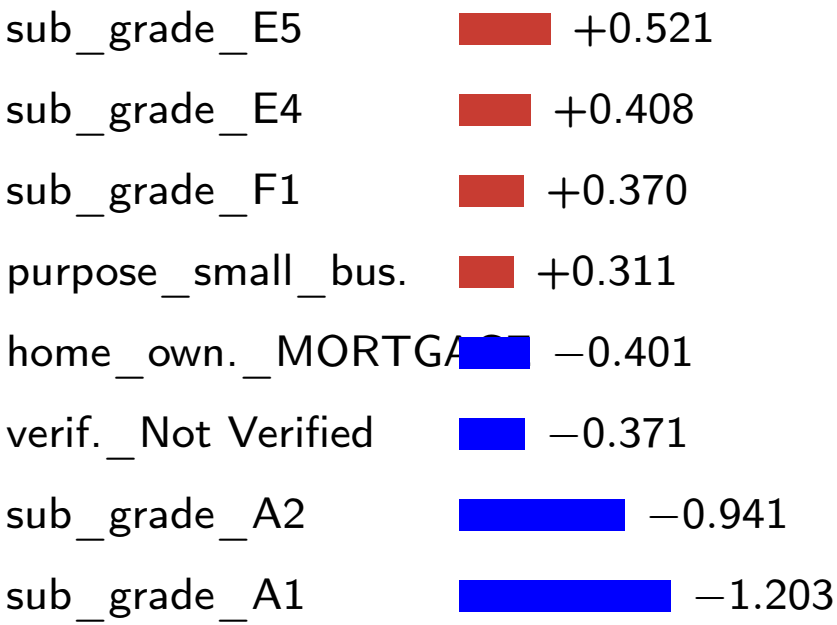
Decision: Logistic L2

- Beats baseline: +0.022 AUC
- Same level as HistGB; no overfitting, fast to train
- Each coefficient is easy to explain

Step 8 Details: Logistic Coefficients

Each coefficient shows how a feature moves default risk: positive = higher risk, negative = lower risk. The test set is used only once, after all choices are made on validation.

Top coefficients (Logistic L2):



Step 8 Details: Known Limits

- Model covers 36-month loans only; test has only 3 months of late 2015
- 32 columns are empty in train; 17 of them only start in val/test
- "Months since event" columns were removed ($>30\%$ missing) though missing may mean "never happened"
- `issue_d` is used in place of application date; only accepted loans exist in the data
- RF and HistGB tuning grids hit the edge of the search space
- One-hot gives 108 sparse dummies: fits Logistic well, harder for trees

Experiment 1: Ablation, Model without `sub_grade`

Remove Lending Club's own grade: 153 → 118 features. Is the model still better?

Test ROC-AUC (axis from 0.66)

L2 (+sub_grade) 0.7039

L2 (no sub_grade) 0.6879

Baseline `sub_grade` 0.6818

- **+0.0075** no sub_grade vs baseline, CI [+0.003, +0.012]
- **+0.0150** value added by sub_grade, CI [+0.013, +0.017]

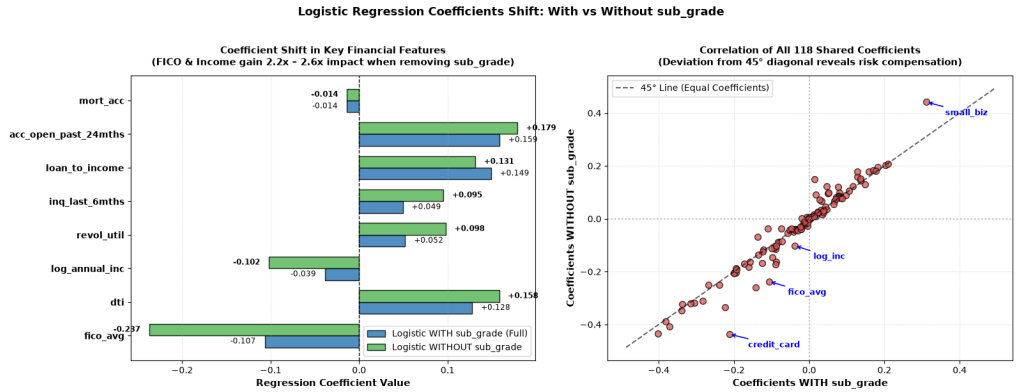


Figure: Coefficient shift when `sub_grade` is removed

`sub_grade` absorbs the risk signal; without it, weight moves back to FICO, income and credit use (1.2×–2.6×)

Experiment 1 Details: Design

- Question: is Logistic still better than the baseline without `sub_grade`?
- `sub_grade` is Lending Club's own grade (A1 – G5)
- Remove all 35 `sub_grade` dummy columns: 153 $\rightarrow$ 118 features
- Same setting: Logistic L2, $C = 1.0$, same train / val / test

experiences/exp_logistic_subgrade.ipynb (excerpt)

```
idx_no_sg = [i for i, c in enumerate(feature_names) if "sub_grade" not in c]

lr_nosg = LogisticRegression(C=1.0, max_iter=1000, random_state=SEED, solver="lbfgs")
lr_nosg.fit(X_train[:, idx_no_sg], y_train)
p_va_nosg = lr_nosg.predict_proba(X_val[:, idx_no_sg])[:, 1]
```

Experiment 1 Details: Results (val and test)

Table: Validation set (92,631 loans)

Model	Features	AUC train	AUC val	Gini val	KS val
Baseline sub_grade	1	0.6565	0.6815	36.31%	27.02%
Logistic L2 (+sub_grade)	153	0.6873	0.7040	40.79%	29.94%
Logistic L2 (no sub_grade)	118	0.6775	0.6890	37.80%	27.68%

Table: Out-of-time test set (88,669 loans)

Model	Test AUC	Δ vs baseline	Gini	KS
Baseline sub_grade	0.6818	—	36.36%	26.45%
Logistic L2 (+sub_grade)	0.7039	+0.0221	40.79%	29.87%
Logistic L2 (no sub_grade)	0.6879	+0.0061	37.58%	27.34%

Experiment 1 Details: ROC Curves

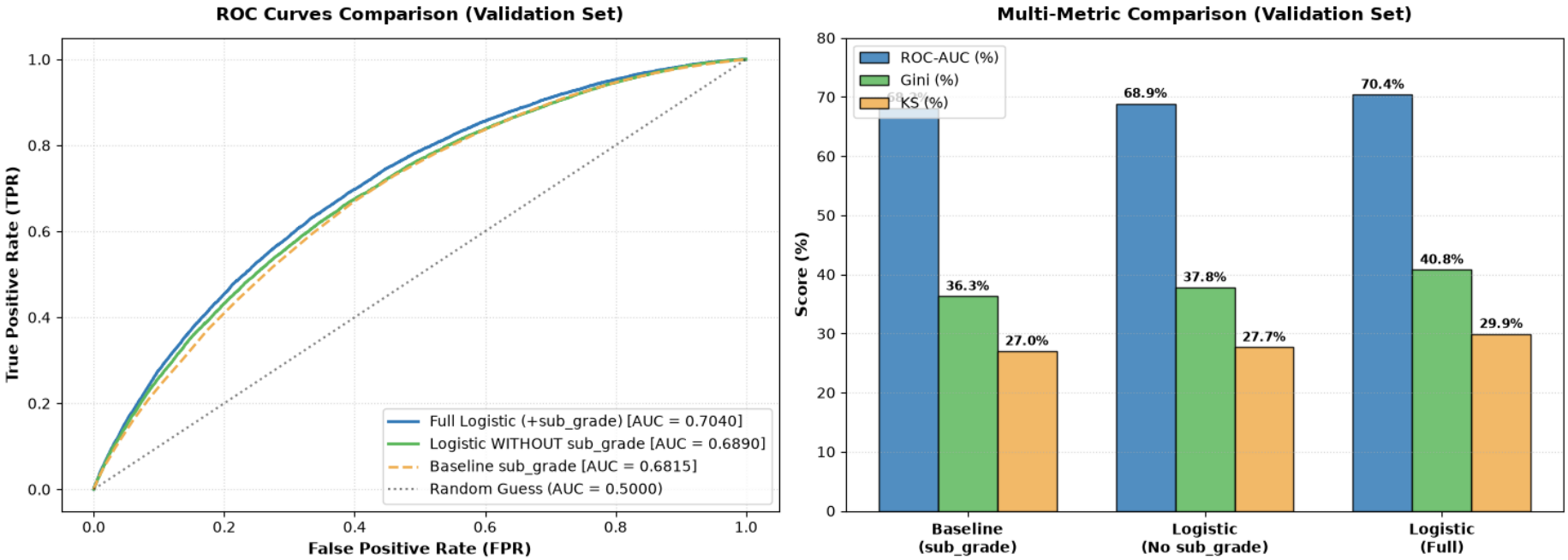


Figure: ROC curves and metric bars on validation (experiment output)

Experiment 1 Details: Bootstrap 95% CI

$B = 1{,}000$ resamples of the validation set.

Table: ΔAUC on validation

Comparison	ΔAUC	95% CI	Result
No sub_grade − Baseline	+0.0075	[+0.0033, +0.0117]	Better than baseline
Full − No sub_grade	+0.0150	[+0.0133, +0.0167]	sub_grade adds value

```
for _ in range(B):
    idx = np.random.randint(0, n_val, size=n_val)
    y_b = y_val[idx]
    diff_nosg_vs_base.append(roc_auc_score(y_b, p_va_nosg[idx]) - roc_auc_score(y_b, p_va_base[idx]))
    diff_full_vs_nosg.append(roc_auc_score(y_b, p_va_full[idx]) - roc_auc_score(y_b, p_va_nosg[idx]))
ci = np.percentile(diff_nosg_vs_base, [2.5, 97.5])
```

Experiment 1 Details: Coefficient Shift

With `sub_grade`: the grade takes most of the risk signal, so other weights are small. Without it: the weight moves back to the borrower's own data.

Table: Logistic coefficients: with vs without sub_grade

Feature	With	Without	Change
fico_avg	−0.107	−0.237	2.22×
log_annual_inc	−0.039	−0.102	2.64×
revol_util	+0.052	+0.098	1.88×
inq_last_6mths	+0.049	+0.095	1.92×
dti	+0.128	+0.158	1.24×

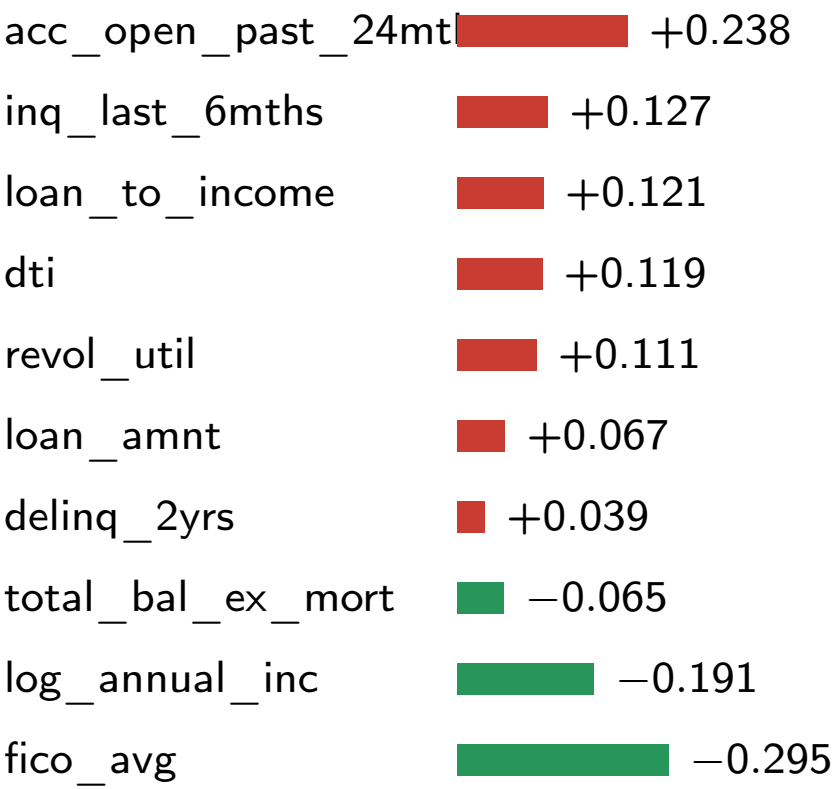
All 153 coefficients are saved in `experiences/coefficients_comparison.csv`.

Experiment 2: Compact Model and Credit Scorecard

12 core features, no `sub_grade`, turned into a 300–850 style score

- **153** $\rightarrow$ **30** columns (-80%); test ROC-AUC **0.673** (Gini 34.6%); score range 494–757 (validation)

Weights of 10 numeric features:



Score scaling:

- Base score 600 at odds 5 : 1
- PDO: 20 points
- Example cut-off: 615
- Higher score = lower risk

2 categorical features:

`home_ownership`, `purpose`

Fewer inputs: cheaper data collection, easy to explain to a loan officer

Experiment 2 Details: 12 Core Features and Performance

- Goal: a smaller model that is cheaper to collect data for; no `sub_grade`
- 10 numeric + 2 categorical features $\rightarrow$ 30 columns after one-hot

10 numeric: fico_avg, dti, log_annual_inc, loan_to_income, revol_util, inq_last_6mths, acc_open_past_24mths, total_bal_ex_mort, delinq_2yrs, loan_amnt · **2 categorical:** home_ownership, purpose

Table: Compact model vs full model

Model	Columns	Val AUC	Test AUC	Test Gini	Test KS
Logistic L2 full	153	0.7040	0.7039	40.79%	29.87%
Compact 12 features	30	0.6750	0.6730	34.60%	25.07%

80% fewer columns (153 $\rightarrow$ 30); keeps 84.8% of the full model's Gini (34.60% / 40.79%).

Experiment 2 Details: Scorecard Scaling

- **600**: base score at odds 5 : 1 (good : bad)
- **20**: PDO, points to double the odds
- **494 – 757**: score range in data
- **615**: example approval cut-off

Score = offset + factor $\times$ ln(odds); factor = PDO / ln(2); offset = 600 − factor $\times$ ln(5). Higher score = lower risk.

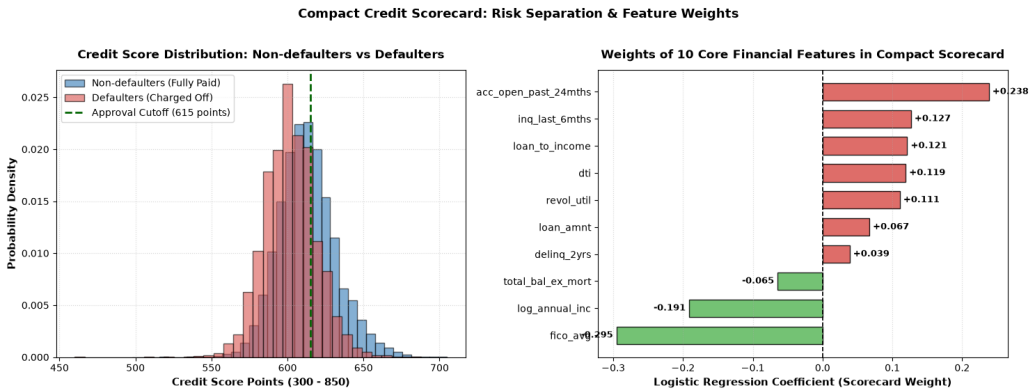


Figure: Score distribution (good vs bad) and weights of the 10 numeric features

Experiment 3: Credit Score Distribution, Good vs Bad Borrowers

Compact 12-feature scorecard, test set, 88,669 loans

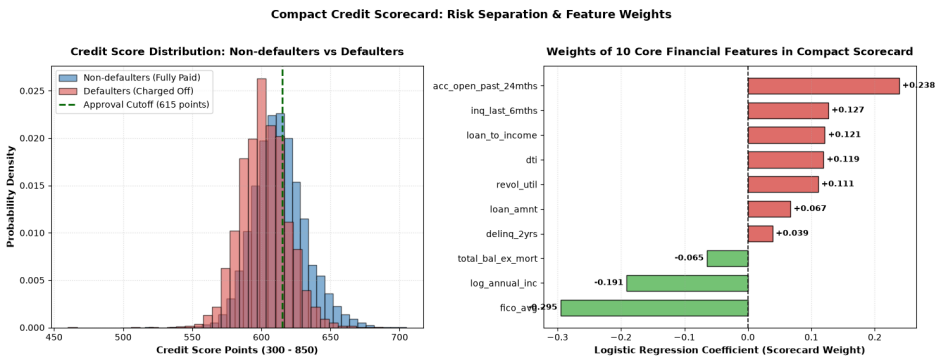


Figure: Score distribution (left) and feature weights (right)

Default rate by band (avg 14.8%)

$<$ 590 Reject	30.2%
590–614 Review	17.3%
615–639 Approve	8.7%
$\geq$ 640 Approve	3.3%

9$\times$ default gap: lowest vs highest band

25.1% test KS

Experiment 3 Details: Score Bands (val and test)

Cut-offs are set on the validation score distribution, then checked on test. Score = offset − factor × ln(odds), PDO = 20, base 600 at odds 5 : 1.

Table: Validation (92,631 loans); score min 494, max 757, mean 610.7

Band	Action	Loans	Share	Default rate	Mean score
< 590	Auto reject	11,279	12.2%	29.38%	580.8
590 – 614	Manual review	44,394	47.9%	17.08%	603.0
615 – 639	Approve with terms	30,379	32.8%	8.32%	624.5
≥ 640	Auto approve	6,579	7.1%	3.45%	650.4

Table: Out-of-time test (88,669 loans); average default rate 14.82%

Band	Action	Loans	Share	Default rate	Mean score
< 590	Auto reject	10,145	11.4%	30.18%	581.0
590 – 614	Manual review	42,171	47.6%	17.28%	603.2
615 – 639	Approve with terms	29,671	33.5%	8.66%	624.5
≥ 640	Auto approve	6,682	7.5%	3.26%	650.7

Experiment 3 Details: Code

experiences/exp_compact_12_features.ipynb (excerpt)

```
factor = PDO / np.log(2)
offset = BASE_SCORE + factor * np.log(1.0 / BASE_ODDS)
credit_scores_test = np.clip(offset - factor * np.log(odds_test), 300, 850).round().astype(int)

bins_score = [300, 590, 615, 640, 850]
df["score_band"] = pd.cut(df["credit_score"], bins=bins_score, labels=labels_score, right=False)
pt = df.groupby("score_band", observed=True).agg(
    default_rate=("actual_default", "mean"),
    mean_score=("credit_score", "mean"))
```